

NARRATIVE REPORT

Student: Sulit, Zachary Lorenzo Contact: 09271766263		Company: Centervert, LLC Contact: ardi@centervert.com	
DATE & TIME	NO. OF HOURS	ACTIVITIES / TASKS	LEARNING
5/6/2026	8 hrs	What did you do that day? Completed company onboarding, established the local development environment, and began the initial phase of ground-truth data preparation by annotating structural PDF drawings. Quality of Experience: The experience was highly technical and engaging. It provided a clear, practical view of how raw industrial data is ingested and prepped before hitting the machine learning pipeline. Steps or the Flow: 1. Onboarding & Synchronization: Attended alignment meetings with the Machine Learning and QA/Annotation teams to understand the overarching detection pipeline. 2. Environment & Infrastructure Setup: Configured local development environments, secured access to the private company GitHub repository, and reviewed technical documentation to understand the backend infrastructure. 3. Tool Familiarization: Explored the core tools used in the workflow, including AI-assisted tools like Codex CLI to navigate scripts and understand repository structures. 4. Structural PDF Annotation: Began the hands-on process of reviewing actual drawing files. This involved	What did I learn? During this eight-hour sprint, I dove into the foundational, often-overlooked stages of applied machine learning and enterprise software engineering. I experienced firsthand that machine learning goes far beyond writing algorithms; it relies heavily on the precise, labor-intensive work of creating high-quality ground-truth data. By taking raw, unstructured industrial data, specifically structural PDFs, and translating them into structured, machine-readable JSON formats, I actively bridged the gap between civil engineering and computer science. I learned how to encode physical, real-world engineering concepts like beam and column layouts into spatial bounding boxes that an AI model can successfully process. Additionally, I mastered the mechanics of establishing a professional development environment, navigating private enterprise repositories, syncing with cross-functional teams, and leveraging modern AI-assisted workflows like the Codex CLI to decipher complex backend infrastructure. Describe if the task was already taught in school or something new.

		identifying relevant structural elements (like beams and column layouts), marking their boundaries accurately, and preparing the annotation results to be exported as structured JSON ground-truth data.	While formal education likely provided the theoretical foundation for these tasks, the practical application I engaged in is almost entirely new compared to a standard academic curriculum. Universities typically focus on the mathematics behind machine learning models, basic version control, and how to train algorithms using perfectly clean, pre-packaged datasets. Academia rarely prepares students for the "cold start" problem of building a dataset entirely from scratch. The messy reality of extracting data from industrial PDFs, annotating it manually, and designing the precise JSON schemas to hold that ground-truth data is a purely industry-driven experience. Unless my specific degree program included a specialized industry practicum focused on data annotation and enterprise infrastructure, dealing with these complex local environments and cutting-edge CLI tools represents a completely novel workflow that extends far beyond standard coursework. If not related in the school, discuss the importance of the learning that it should be taught in the school. The gap between academic machine learning theory and industry reality is a well-known bottleneck in the tech sector, and the skills I practiced today are critical enough that they should absolutely be integrated into university curricula. Data scientists and machine learning engineers spend the vast majority of their time finding, cleaning, and annotating
--	--	---	--

			data, leaving only a small fraction for actual model tuning. Schools currently invert this ratio, graduating students who know how to optimize a neural network but struggle when handed a folder of raw, unstructured PDFs. Teaching ground-truth data generation and cross-disciplinary problem-solving would make graduates immediately valuable to employers, as real-world AI must be meticulously applied to specific, complex domains such as construction. Furthermore, integrating professional-grade developer operations, such as environment configuration, dependency management, and secure repository access, into coursework would drastically reduce the friction junior engineers face, significantly accelerating their ability to contribute to production environments.
5/7/2026	8hrs	What did you do that day? Continued ground-truth preparation by annotating a 75-page structural PDF set (AWS PHL 1b). Later, configured the QA environments to review a pipeline of Pull Requests (PRs) and attended a QA verification meeting to align on expected detection outcomes. Quality of Experience: The shift from manual annotation to active QA and code review was challenging but highly rewarding. It provided a clear picture of how annotation data directly informs the validation of repository changes. Steps or the Flow:	What did I learn? During this second eight-hour shift, I expanded my understanding of the machine learning lifecycle by connecting data preparation directly to model validation and code remediation. I learned that annotation is not just an isolated task, but a benchmark used to evaluate code changes through automated workflows. Specifically, I gained deep technical insights into error analysis by diagnosing a critical failure in an active pull request, PR51, where vector-based pages with dense, overlapping text labels triggered a spike in false positives. I learned how to resolve these spatial anomalies by

		1. PDF Annotation: Systematically reviewed 75 pages of the AWS PHL 1b structural drawings, marking relevant structural elements to export as structured JSON ground truth. 2. QA Environment Setup: Configured the local testing environments to pull specific PR branches from the GitHub repository for validation. 3. PR Review & Error Analysis: Ran benchmarking scripts against pending PRs to evaluate model behavior. 4. PR51 Remediation: Investigated PR51 using <code>diff_candidate_runs.py</code> and noted unusually high error scores, specifically a spike in false positives. The drop in precision was caused by a thresholding calibration issue where the detection engine misclassified dense, overlapping text labels as structural beams in vector-based pages. I implemented configuration changes to adjust the confidence thresholds and geometric boundaries (using Shapely logic), successfully reducing the duplicate detections before pushing the fix. 5. QA Verification Meeting: Concluded the day with a 6:00 PM - 8:00 PM alignment meeting to review these specific PR outcomes and finalize QA protocols with the engineering team.	adjusting confidence thresholds and implementing geometric boundary logic using the Shapely library, successfully reducing duplicate detections. Finally, by managing pull requests and participating in a two-hour evening verification meeting, I learned the collaborative mechanics of how engineering teams align on quality assurance protocols before pushing code to production. Describe if the task was already taught in school or something new. This shift introduced highly advanced, industry-specific engineering workflows that are almost completely new compared to what is taught in school. While university projects might touch upon basics like git commits or standard data manipulation, they rarely cover the complexities of cross-functional QA operations. The process of pulling live pull request branches to benchmark a model against newly annotated ground-truth data is uniquely enterprise-driven. Furthermore, debugging production-level scripts like <code>diff_candidate_runs.py</code> to perform spatial error analysis, and using programmatic geometric tools like Shapely to resolve real-world thresholding and calibration issues, are practical engineering skills that extend far beyond standard academic assignments. If not related in the school, discuss the importance of the learning that it should be taught in the school.
--	--	--	---

			The skills gained during this day are vital for a modern engineer and highlight gaps that should be addressed in university curricula. In academia, grading is typically binary based on code execution, but in the industry, the focus is on performance degradation, edge cases, and regression testing. Schools should teach students how to systematically diagnose model failures, such as tracking precision drops caused by noisy text, because debugging data-driven systems is fundamentally different from debugging traditional software. Additionally, introducing students to pull request reviews, benchmarking pipelines, and rigorous engineering alignment meetings would better prepare graduates for the highly collaborative code-maintenance environments they will navigate on day one of their careers.
5/8/2026	8hrs	What did you do that day? Continued ground-truth data preparation by annotating the "Hope Landing" and "CFMC" structural PDF sets. During the benchmarking process, I discovered a high volume of detection errors and manually traced them back through the backend pipeline to identify the root causes. Quality of Experience: The experience was highly analytical and practical. Moving beyond standard annotation to actively debug the detection architecture provided a deep, satisfying dive into production-level machine learning troubleshooting.	What did I learn? During this sprint, I moved from data generation into advanced architectural troubleshooting, learning how to systematically trace model degradation through a complex machine learning pipeline. I gained practical insight into how a multi-stage detection system can fail downstream and learned how to isolate errors between geometric candidate generation and subsequent classification models. Specifically, I learned the mechanics of document archetype routing, analyzing whether a page was misclassified and sent to the wrong specialized model, such as routing an arc-heavy page to a Thin

		Steps or the Flow: 1. PDF Annotation: Annotated the Hope Landing and CFMC drawing files, identifying and marking structural elements to export as structured ground-truth data. 2. Benchmarking & Error Discovery: Ran the Python evaluation workflows against the newly annotated data. The results showed a significant number of false positives and incorrect classifications. 3. Pipeline Tracing: I manually traced these errors through the system's architecture, specifically referencing the workflow 4. Component Analysis: I analyzed whether the initial pdf_engine.py (Geometric Candidate Generator) failed to capture the elements or if the issue occurred downstream. For beam candidates, I checked if the PDF Cluster Router was misclassifying the document archetype. For example, I looked into whether it was incorrectly routing a bold, arc-heavy page to the Thin Specialist (C0, C4) instead of the Bold Specialist (C1, C2, C3). 5. Threshold Verification: Finally, I reviewed the per-source thresholding and geometric deduplication phases to see if the LightGBM models were scoring candidates incorrectly before generating the final validated output.	Specialist instead of a Bold Specialist. Finally, I learned how to evaluate the final thresholding and geometric deduplication phases, verifying whether a LightGBM scoring model or the post-processing filters were responsible for generating a high volume of false positives. Describe if the task was already taught in school or something new. This deep dive into pipeline tracing and component-level error analysis is entirely new compared to standard academic coursework. In an educational setting, machine learning assignments usually involve a single model where I tweak hyperparameters or change layers to improve accuracy metrics. School rarely exposes students to complex, multi-stage production architectures where an initial script handles geometric candidate generation, an intermediate router assigns data to specialized sub-models based on document archetype, and a final LightGBM model handles thresholding. Manually tracing data anomalies through these independent, interconnected backend stages to debug a structural failure is a highly specialized industry practice. If not related in the school, discuss the importance of the learning that it should be taught in the school. This type of architectural troubleshooting is incredibly important and should be integrated into university curricula because
--	--	--	---

			real-world AI systems are rarely composed of just a single model. When a production system fails, a data scientist cannot simply retrain the entire network; they must know how to isolate which specific module, router, or post-processing threshold failed. Teaching students how to perform component analysis and trace data flow through a multi-model pipeline prepares them for the actual engineering challenges of industry. Shifting the academic focus from optimizing isolated algorithms to debugging complex pipelines would give students a much more realistic and valuable foundation for their engineering careers.
5/11/2026	8hrs	What did you do that day? Finalized ground-truth annotations for the Phil 102 structural drawing set. Additionally, I identified and escalated a critical "zero detection" bug within the SteelSnap engine across four separate PDFs where the pipeline completely failed to extract elements. Quality of Experience: The experience was highly technical and focused on quality assurance communication. It highlighted the importance of bridging the gap between data evaluation and developer operations by translating visual errors into actionable technical reports. Steps or the Flow: 1. PDF Annotation: Completed the detailed review and annotation of the	What did I learn? During this shift, I learned the critical role of edge-case discovery and technical escalation within a production-level machine learning workflow. I experienced how to handle a catastrophic pipeline failure, specifically a zero-detection bug where the system completely fails to extract elements from a document. I learned how to audit the earliest entry points of a pipeline, checking if an engine like pdf_engine.py failed due to heavy rasterization, unreadable vector layers, or PyMuPDF rendering bugs before data could even reach downstream routers. Crucially, I learned how to translate these visual and behavioral model failures into structured, actionable developer reports, ensuring that the engineering team received precise

		Phil 102 PDF set. I accurately marked the relevant structural elements and exported the final JSON ground-truth data to feed the benchmark runners. 2. Error Discovery (SteelSnap): While running benchmark evaluations, I discovered that four specific PDFs returned a "reads no detection" state, meaning the system output was entirely blank. 3. Root Cause Investigation: I investigated the early stages of the pipeline to see where the data dropped off. I analyzed whether pdf_engine.py (the Geometric Candidate Generator) was failing to process the Input PDF due to heavy rasterization, unreadable vector layers, or PyMuPDF rendering failures, which prevented any candidates from reaching the PDF Cluster Router. 4. Developer Escalation: I compiled a comprehensive bug report for the engineering team. Instead of just noting that the PDFs failed, I documented the exact pipeline stage where feature extraction stalled, ensuring the developers had precise technical context to apply a fix.	architectural context regarding where feature extraction stalled rather than just a vague error notification. Describe if the task was already taught in school or something new. While my university courses likely covered standard data handling and basic error checking, investigating a total pipeline drop-off and managing developer escalation is entirely new. In an academic environment, datasets are deliberately curated to ensure they are readable, and if an error occurs, it usually results in a clear Python stack trace. School rarely prepares me for silent data failures where a script executes completely but outputs an entirely blank state because a third-party library like PyMuPDF failed to parse specific vector layers. Furthermore, the practice of conducting a root-cause investigation across low-level geometric engines and compiling a formal engineering bug report is a professional QA methodology that extends far beyond standard classroom assignments. If not related in the school, discuss the importance of the learning that it should be taught in the school. This learning is incredibly important and should be a staple in university curricula because handling unexpected, silent data corruption is a daily reality in data science. Graduates who only know how to work with clean, pre-processed academic datasets often struggle
--	--	---	---

			when an enterprise application fails silently in production due to real-world file variances like heavy rasterization. Teaching students how to systematically audit the ingestion layer of an architecture and isolate rendering failures builds essential debugging resilience. Additionally, learning how to write highly detailed technical escalations bridges the communication gap between data analysts and backend infrastructure developers, a cross-functional skill that is critical for the success of any commercial engineering team.
5/12/2022 6	8hrs	What did you do that day? Completed ground-truth annotation for the "Safeindust" structural PDF set and identified multiple complex layout issues. I then transitioned to code review and data analysis, exporting all annotations for an Exploratory Data Analysis (EDA) deep dive and generating an automated performance report for my supervisor. Quality of Experience: The experience was highly analytical and heavily focused on applied data science. Moving from qualitative visual checks to quantitative statistical analysis of model performance was incredibly fulfilling. Steps or the Flow: 1. PDF Annotation: Finished annotating the Safeindust PDFs. I documented several edge-case drawing conventions that caused detection errors.	What did I learn? During this shift, I advanced my skills in applied data science by shifting from qualitative visual data checks to quantitative statistical analysis. I learned how to detect model overfitting within an enterprise repository by executing a standard deviation analysis on error metrics across varied document archetypes. Through this, I grasped how a model configuration can inadvertently optimize for specific drawing layouts while failing on others. Additionally, I learned how to process and analyze large-scale spatial data by exporting validated annotations into JSON, parsing them with Pandas and PyArrow, and conducting an exploratory data analysis to map the spatial distribution and bounding box sizes of beams and columns. Finally, I learned to automate engineering workflows by utilizing Python scripts to aggregate complex spatial segmentation results and

	2. GitHub PR Review & Overfitting Check: I reviewed recent Pull Request (PR) changes in the repository. I noticed a potential overfitting issue where a model configuration seemed to perform perfectly on specific drawing types but failed on others. To quantify this, I conducted a Standard Deviation (STD) analysis on the error metrics across different PDF archetypes. I formalized these statistical findings and submitted them for the PR review. 3. Data Export & EDA: I exported all the validated annotated PDFs into structured JSON files. Using Pandas and PyArrow, I parsed this data to perform Exploratory Data Analysis (EDA). The focus was on spatial segmentation and mapping the distribution of bounding boxes for beams and columns across different page formats. 4. Automated Reporting: I used an internal automated tool (a Python-based Pi script) to aggregate the EDA segmentation results and the PR error analysis into a cohesive data science report. I submitted this directly to my immediate supervisor.	PR error metrics into a cohesive, structured report for leadership. Describe if the task was already taught in school or something new. This day combined elements taught in my data science curriculum with highly advanced, novel industry applications. My university coursework definitely introduced me to exploratory data analysis, data manipulation using Pandas, and basic descriptive statistics like standard deviation. However, applying these concepts to spatial segmentation analysis of bounding boxes on engineering schematics is entirely new. Moreover, using statistical distribution checks specifically to audit version-controlled pull requests for overfitting, or parsing complex JSON metadata with PyArrow within an active production repository, represents an industrial data engineering workflow that goes far beyond standard textbook assignments. If not related in the school, discuss the importance of the learning that it should be taught in the school. This workflow highlights a critical, practical intersection of data science and software engineering that belongs in university classrooms. Academic programs often teach exploratory data analysis on clean, flat tabular datasets like house prices or passenger demographics, rather than complex spatial annotations extracted from unstructured files.
--	--	--

			Teaching students how to use statistical analysis to evaluate live model regressions and pull requests would bridge the massive gap between theoretical modeling and production-level validation. Furthermore, learning how to automatically compile engineering metrics into automated executive reports prepares students for the reporting expectations of corporate tech environments, turning them into data professionals who can communicate technical findings clearly to supervisors.
5/13/2026	8hrs	What did you do that day? Completed the ground-truth annotations for the "Tuscarora dc" structural PDF set. Afterward, I transitioned to system validation by joining a brief meeting focused on live demo testing for Project SteelSnap. Quality of Experience: The experience was highly engaging. Transitioning directly from raw data annotation to watching the engine run in a live demonstration provided a comprehensive view of the product lifecycle. Steps or the Flow: 1. Data Preparation: Finalized the visual inspection and annotation process for the Tuscarora dc PDF. I marked the necessary structural elements and exported the final results into structured JSON format to feed the benchmark runners. 2. Demo Synchronization: Joined the SteelSnap demo meeting with the	What did I learn? During this shift, I learned how the entire data-to-production lifecycle culminates in my ground-truth data directly validating a live machine learning system. By transitioning from annotating the Tuscarora dc PDF set to observing a live demonstration of Project SteelSnap, I gained critical insights into model deployment and front-end synchronization. Specifically, I learned how ONNX Runtime object detection models and downstream LightGBM specialist algorithms process raw structural drawings dynamically in real time. I also learned the importance of geometric mapping validation, observing how the backend detection engine translates coordinates into visual bounding boxes within a staging user interface without causing front-end clutter or rendering alignment errors.

		team. The objective was to evaluate how the final detection outputs, specifically the validated beams and columns, aligned with the display coordinates in the staging interface. 3. Live Testing Observation: We observed the pipeline processing structural PDFs in real time. We specifically watched how the ONNX Runtime object detection models and the LightGBM specialists handled the inputs dynamically, checking for visual accuracy and ensuring the geometric bounding boxes were mapped cleanly without front-end clutter.	Describe if the task was already taught in school or something new. While university courses provide the foundations for model training and exporting such as saving models or working with standard dataset formats this live, system-wide validation process is completely new. Academic projects typically end once a model achieves a certain accuracy score on a test set, rarely advancing to a staging environment where the model runs in real time via an optimized execution engine like ONNX Runtime. Participating in a live deployment demo, verifying real-time pipeline latency, and checking how backend algorithmic coordinates align dynamically with front-end interface components are purely industry-driven engineering experiences that are not replicated in a classroom setting. If not related in the school, discuss the importance of the learning that it should be taught in the school. This experience underscores a massive gap in traditional tech education and highlights why end-to-end product validation should be integrated into university curricula. Students often graduate believing that a machine learning engineer's only job is to maximize mathematical metrics, disconnected from how users interact with those predictions. Teaching students how models perform in real-time staging environments and how inference engines like ONNX optimize
--	--	---	---

			deployment would prepare them for the computational constraints of the industry. Furthermore, learning how data flows across the entire pipeline, from raw JSON annotations to a live user interface, teaches students to build models that are not just mathematically sound, but also functionally viable in a real-world commercial application.
5/14/2026	8hrs	What did you do that day? Participated in developer testing and Project SteelSnap synchronization meetings. I contributed to a specific classification ticket and documented system pipeline bottlenecks where detection processing stalled. Quality of Experience: The experience was highly collaborative and system-oriented. It shifted the focus from static data annotation to live application performance and cross-departmental task management. Steps or the Flow: 1. Dev Testing & Bug Replication: Attended the Discord testing meeting starting at 6:00 PM. I actively tested the web interface and documented multiple application crashes. 2. Pipeline Bottleneck Analysis: Investigated the detection pipeline hanging between 70 and 80% percent completion. The system was stalled, indicating a potential rendering loop or memory overload within the backend processing phases.	What did I learn? During this eight-hour shift, I learned the critical skills of application stress testing, performance profiling, and cross-departmental task tracking within a live production environment. I gained deep insight into complex backend operations by investigating pipeline bottlenecks where the system stalled mid-execution, learning to recognize the technical signatures of rendering loops and memory overloads. Additionally, I developed a practical understanding of enterprise project management by handling ticket CEN-637 within the Linear app, specifically formatting and documenting decision tree classification logic for the Gatekeeper module. Finally, I learned the value of high-fidelity team communication by translating replicated web interface crashes and pipeline freezes into clear, actionable data for the core engineering team during my synchronization meetings.

		3. Cross-Departmental Ticketing: Contributed to ticket CEN-637 regarding the "Gatekeeper receive + classify with decision tree" feature. I formatted the findings and uploaded the ticket to the Linear app so other departments could review the classification logic. 4. SteelSnap Synchronization: Concluded the shift with the Project SteelSnap meeting from 9:00 PM to 10:00 PM to formally report all replicated crashes and pipeline stalls to the core development team.	Describe if the task was already taught in school or something new. While my academic courses likely covered the theoretical concepts behind decision trees and basic software debugging, this collaborative, systems-level testing workflow is completely new. In a university environment, programming assignments are self-contained, and code failures usually present themselves as predictable runtime exceptions with clear stack traces. School rarely prepares me for the messy reality of diagnosing an entire data pipeline that silently hangs at eighty percent completion without throwing an explicit error. Managing cross-departmental tickets using professional agile infrastructure like Linear, while executing interactive, multi-hour replication sessions over team communication platforms, represents a highly practical engineering methodology unique to the tech industry. If not related in the school, discuss the importance of the learning that it should be taught in the school. This workflow highlights a major operational gap that should be addressed in university curricula, because professional developers spend far more time maintaining, debugging, and tracking existing pipelines than writing new algorithms from scratch. Computer science programs heavily emphasize building pristine, isolated projects, leaving graduates underprepared for real-world issues
--	--	---	---

			like memory leaks or asynchronous processing stalls. Integrating modern issue-tracking software and collaborative project boards into group coursework would teach students how to properly document system failures for cross-functional teams. Shifting academic milestones to include rigorous pipeline profiling and structured bug replication would create engineering graduates who can instantly diagnose large-scale application crashes and communicate effectively under operational pressure.
5/15/2026	8 hrs	What did you do that day? Participated in a Daily Stand-up Meeting (DSM) and a broader team alignment for Project SteelSnap. To improve the engine's detection success rate, we coordinated the distribution of 50 massive structural PDFs (each over 100 pages) across the intern team for diverse use-case testing. Quality of Experience: The experience was highly collaborative and strategic. It provided an excellent perspective on project management, workload balancing, and the logistics of scaling up data testing to meet tight production deadlines. Steps or the Flow: 1. Agile Synchronization: Held a morning DSM followed by a Project SteelSnap team meeting to discuss new tasks and outline strategies for increasing detection accuracy.	What did I learn? In this session, I learned the logistical and strategic realities of scaling up machine learning operations under tight production deadlines. Through the morning Daily Stand-up Meeting and team alignment, I observed how agile frameworks function in a corporate setting to keep cross-functional teams synchronized. I learned how to execute workload distribution strategies by splitting a massive testing batch of fifty 100-page structural PDFs among a team to prevent pipeline bottlenecks. Furthermore, I learned the technical value of diverse use-case testing, specifically how exposing the PDF Cluster Router and LightGBM specialists to varied document archetypes and dense layout conventions provides the critical, unseen data points required for subsequent model calibration and accuracy optimization.

		2. Workload Strategy: To handle the immediate deadline without bottlenecking the pipeline, we evenly distributed a testing batch of 50 large-scale PDFs among the interns. Each document contained at least 100 pages of complex structural drawings. 3. Diverse Use-Case Testing: We deliberately selected varying PDF archetypes to serve as different use cases. The goal was to run these through the pipeline to see how the PDF Cluster Router and the LightGBM specialists handled unseen drawing conventions and diverse layout densities. 4. Detection Evaluation: Began running the assigned batches through the detection workflows, noting where the engine succeeded and where it struggled, allowing the team to gather varied data points for the next model calibration.	Describe if the task was already taught in school or something new. This operational experience is entirely new compared to standard academic coursework. While universities often assign group projects, they rarely simulate authentic agile ceremonies like Daily Stand-ups or require strategic workload balancing across a team to meet immediate enterprise deadlines. In a classroom, datasets are sized for individual completion, meaning I rarely deal with the sheer scale of managing thousands of document pages simultaneously. The practice of systematically selecting diverse, unseen data archetypes to intentionally stress-test how routers and specialized algorithms handle layout variance is an industry-level quality assurance strategy that goes far beyond traditional textbook exercises.
5/18/2026	8hrs	What did you do that day? Participated in the Daily Stand-up Meeting (DSM), conducted rigorous detection testing, and documented the results for machine learning improvements. I also performed Exploratory Data Analysis (EDA) on the first batch of 10 PDFs and prepared the structured JSON files for spatial mapping and segmentation. Quality of Experience: The experience was highly structured and data-centric. Bridging the gap between manual visual testing and programmatic spatial analysis provided	• What did you learn? During this shift, I learned how structured feedback loops and programmatic data analysis directly drive machine learning model optimization. By documenting how the detection engine handled varying drawing conventions, I gained a practical understanding of how raw testing data translates into actionable parameters for adjusting confidence thresholds and fine-tuning LightGBM specialist models. I also advanced my data engineering skills by extracting live pipeline outputs and converting them into structured, query-ready JSON files with properly aligned bounding box coordinates. Finally, I learned how to execute spatial

	a deep understanding of how feedback loops drive model improvements. Steps or the Flow: 1. Agile Alignment: Started the shift with the DSM to align on daily goals, specifically focusing on the new detection testing batches. 2. Detection Testing & Documentation: Ran the structural PDFs through the detection pipeline. I evaluated how the engine handled different drawing conventions and meticulously documented the detection behaviors. This documentation was formatted specifically for the ML engineering team to help them adjust confidence thresholds and improve the LightGBM specialist models. 3. Data Preparation (JSON): Extracted the detection outputs and formatted them into structured JSON files, ensuring that all bounding box coordinates and element classifications were properly aligned for downstream analysis. 4. Exploratory Data Analysis (EDA): Loaded the JSON data for the first 10 PDFs using Pandas and PyArrow. I performed EDA focused on spatial segmentation, mapping the distribution of structural elements across the pages to identify geometric patterns and potential areas where the model struggles with dense clusters.	segmentation via exploratory data analysis, utilizing Pandas and PyArrow to map geometric distributions and locate specific structural density clusters where object detection models face performance degradation. • Describe if the task was already taught in school or something new. This day presented a hybrid of familiar academic concepts and entirely new industry workflows. My data science coursework covered the basics of exploratory data analysis, data manipulation with Pandas, and general machine learning theory. However, the specific application of these tools to perform spatial segmentation and bounding box distribution mapping on engineering schematics is completely new to me. In a traditional university setting, I typically performed data analysis on flat tabular datasets rather than hierarchically structured JSON files representing complex physical coordinates. Furthermore, the practice of using live testing results to create an immediate feedback loop for tuning downstream production models like LightGBM specialists represents a mature, collaborative engineering methodology that extends far beyond my typical classroom assignments. • If not related in the school, discuss the importance of the learning that it should be taught in the school The ability to perform spatial analysis and connect data
--	--	--

			evaluation directly to model improvement is an essential skill that should be fully integrated into university curricula. Academic programs frequently overemphasize modeling on pre-cleaned data while neglecting the critical infrastructure of machine learning operations, such as how testing documentation actually informs threshold calibration in production. Teaching students how to parse complex JSON coordinates and map spatial distributions would better prepare them for modern data roles, as industries increasingly rely on computer vision and spatial intelligence rather than just standard tabular data. By introducing these production-level feedback loops and advanced spatial exploratory data analysis techniques into coursework, universities would graduate data scientists who can immediately bridge the gap between initial data ingestion and continuous model optimization.
5/19/2026	8hrs	What did you do that day? Attended the machine learning progress meeting, conducted batch detection testing on 32 PDFs, compiled a technical analysis report, and generated specialized ground-truth annotations targeting the Bold Specialist beam routing cluster. Quality of Experience: Highly technical and specialized. Focusing on a specific sub-model within the architecture provided a deeper understanding of how targeted	• What did you learn? During this eight-hour shift, I learned the strategic importance of targeted data engineering and localized model calibration within a multi-stage machine learning pipeline. By focusing my manual annotation efforts strictly on bold and arc-heavy structural drawings, I learned how to isolate and curate data for a specific sub-model cluster, specifically the specialist bold r2 LightGBM model, to ensure its isotonic calibration functions correctly. I also expanded my technical operations skillset by executing a sharded benchmarking

		data feeds improve complex machine learning pipelines. Steps or the Flow: 1. ML Progress Meeting: Synchronized with the engineering team to review recent evaluation metrics, discuss the routing efficiency of the PDF Cluster Router, and align on upcoming calibration goals. 2. Targeted Ground-Truth Generation (C1, C2, C3): Focused manual annotation efforts strictly on beam elements within clean, bold, and arc-heavy structural drawings. This data is critical for establishing ground truth specifically designed to evaluate and fine-tune the specialist_bold_r2 LightGBM model, ensuring its isotonic calibration performs correctly. 3. Batch Detection Testing: Executed the <code>run_sharded_benchmark.py</code> workflow to run the pipeline across a targeted batch of 32 PDFs. I evaluated how the initial geometric candidate generator processed these specific layouts. 4. Analysis Reporting: Compiled a comprehensive technical analysis report. I aggregated the exported JSON candidate data using Pandas to summarize model errors, false-positive spikes, and the effectiveness of the per-source thresholding and geometric deduplication stages.	workflow to run the pipeline across a targeted batch of 32 PDFs, observing how the geometric candidate generator handles varied layouts. Finally, I learned how to quantitatively evaluate production-level post-processing stages by using Pandas to aggregate exported JSON candidate data, which allowed me to analyze specific model errors, map false-positive spikes, and evaluate the effectiveness of per-source thresholding and geometric deduplication. • Describe if the task was already taught in school or something new. While my university data science coursework provided a solid foundation in using Pandas for data manipulation and introduced general machine learning theories, the highly specialized architectural tasks I performed today are entirely new. In school, machine learning models are typically evaluated as simple, monolithic systems trained on clean, uniform datasets. My academic training did not expose me to advanced concepts like document archetype routing, nor did it cover the necessity of optimizing a specific downstream sub-model using specialized ground-truth subsets. Furthermore, running distributed, sharded benchmark workflows and auditing proprietary post-processing layers, such as geometric deduplication and per-source thresholding, represents an industrial engineering reality that is completely absent from traditional classroom assignments.
--	--	---	---

			• If not related in the school, discuss the importance of the learning that it should be taught in the school The advanced techniques practiced during this shift are critical for production-grade artificial intelligence and represent essential methodologies that should be integrated into university curricula. In the commercial tech sector, enterprise computer vision and document processing systems rarely rely on a single, isolated algorithm; they depend on complex routing pipelines and highly specialized ensembles. Teaching students how to perform targeted data curation to fix a specific failing sub-component, rather than simply retraining an entire system, would prepare them for the modular debugging constraints of the industry. Additionally, concepts like isotonic calibration, sharded evaluation, and geometric post-processing are vital for deploying reliable, scalable models. Introducing these production-level practices in school would ensure graduates understand how to manage complex, interconnected architectures and conduct sophisticated error analysis in professional environments.
5/20/2026	8hrs	What did you do that day? Conducted comprehensive detection testing and validation across 35 structural drawing PDFs. I documented specific failure patterns, compiled a privacy-compliant analysis report, and began triaging a new batch of heavily	• What did you learn? During this eight hour shift, I learned how to systematically categorize and analyze recurring failure patterns within a machine learning detection pipeline to improve overall system stability. By evaluating thirty-five structural PDFs against expected ground truth, I developed a deeper

	vectorized PDFs to assess processing bottlenecks. Quality of Experience: Highly rigorous and forward-looking. Moving from immediate error logging to anticipating pipeline stress with complex vectorized PDFs built a strong foundation in system scalability and performance optimization. Steps or the Flow: 1. Batch Detection Validation: Executed the benchmarking workflow on 35 structural PDFs. I systematically reviewed the generated beam and column outputs against the expected ground truth. 2. Error Pattern Documentation: I documented recurring issues affecting model performance. I specifically categorized failures such as False Positives, Missed Members, Missed Labels, and incomplete annotations. For instance, I traced cases of poor column detection back to potential failures in the Column Geometric Baseline. 3. Reporting: Prepared the corresponding analysis report recording these observed failure patterns and the specific source-data conditions that triggered them. 4. Vectorized PDF Triage: Received an additional distribution of PDFs for the next phase of testing. I focused on identifying heavily vectorized files. I analyzed how complex linework, dense graphical elements, and numerous vector objects might strain pdf_engine.py during ingestion, potentially causing performance	understanding of how source data conditions trigger specific errors like false positives, missed members, and missed labels. I learned to trace poor column detection back to faults in the Column Geometric Baseline, gaining practical insight into how algorithmic thresholds affect physical element extraction. Furthermore, through triaging heavily vectorized documents, I learned to evaluate upstream processing bottlenecks, specifically analyzing how dense linework and intricate vector layers can strain an ingestion engine like pdf_engine.py before the data ever reaches the routing or classification stages. • Describe if the task was already taught in school or something new. This experience introduced highly specialized quality assurance and performance testing workflows that are entirely new compared to my academic training. While my university courses taught me the basic theory of model evaluation metrics such as precision and recall, they never exposed me to the granular process of establishing an error taxonomy to categorize failures like missed labels or missed members. In a school environment, datasets are usually uniform, so I never had to triage raw files based on their internal rendering complexity. Analyzing how heavily vectorized files physically strain an ingestion engine or diagnosing a specific failure in a specialized module like the Column Geometric Baseline represents an infrastructure
--	---	--

		bottlenecks and reducing detection reliability before the candidates even reach the PDF Cluster Router.	focused debugging methodology that extends far beyond standard computer science assignments. ● If not related in the school, discuss the importance of the learning that it should be taught in the school The skills I practiced today are incredibly vital for production level engineering and highlight critical operational gaps that should be addressed in university curricula. In traditional computer science programs, students rarely learn that a model failure can stem from the structural format of the input file itself, such as heavy vectorization causing an ingestion bottleneck. Teaching students how to perform granular error analysis and trace issues back to specific architectural sub components prepares them for the realities of maintaining scalable AI applications. By integrating performance profiling, file triage, and modular system validation into the classroom, universities would graduate data professionals who look beyond global accuracy scores and possess the technical foresight to optimize pipelines for real world data complexities.
5/21/2026	8hrs	What did you do that day? Finalized the detection analysis report for a new batch of 10 structural PDFs and conducted a secondary review of previously evaluated plan sets. I integrated automated benchmarking workflows to quantify exactly how source-data noise and pipeline misclassifications affect the final outputs.	● What did you learn? During this eight-hour shift, I learned how source-data noise and upstream misclassifications cause compounding errors across a multi-stage machine learning pipeline. By evaluating ten structural PDFs with automated benchmarking scripts, I quantified exactly how data noise like excessive vector hatching degrades LightGBM candidate scoring and precision-recall

		Quality of Experience: Highly analytical and systems-oriented. Combining rigorous manual verification with automated Python scripting provided a holistic, end-to-end view of data validation and model performance. Steps or the Flow: 1. Secondary Validation Review: Rechecked previously evaluated plan sets to verify the categorization of detection issues, specifically False Positives, Missed Members, Missed Labels, and incomplete annotations. I paid close attention to source-data anomalies like heavy background noise and stray linework that disrupt geometric candidate generation. 2. Automated Benchmarking: Processed the finalized batch of 10 PDFs using <code>run_sharded_benchmark.py</code> and <code>compare_manual_counts.py</code>. This established updated precision and recall baselines, quantifying the exact degradation in LightGBM candidate scoring caused by data noise. 3. Cluster Router Analysis: Investigated upstream pipeline behavior. I discovered that excessive vector hatching caused the PDF Cluster Router to incorrectly send drawings suited for the Thin Specialist (C0, C4) pipeline into the global fallback model, leading to sudden recall drops. 4. Geometric Deduplication & Thresholding: Analyzed false-positive floods triggered by architectural markups. I documented edge cases where the per-source threshold	baselines. I discovered how this noise can deceive the PDF Cluster Router, causing it to misroute drawings intended for the Thin Specialist pipeline into a global fallback model, which triggers severe drops in recall. Additionally, I learned how to analyze false-positive floods caused by architectural markups, using Shapely to document failures in geometric deduplication and running differential scripts to pinpoint whether missed elements were dropping out during initial geometric candidate generation or during post-processing filters. • Describe if the task was already taught in school or something new. While my university courses introduced the concepts of precision and recall baselines along with general script execution, this complex, end-to-end pipeline validation workflow is entirely new. In academic settings, data science projects rarely involve running multi-layered benchmark scripts to track how upstream routing errors directly impact downstream geometric deduplication. School did not prepare me to diagnose how physical artifacts, such as architectural markups or excessive vector hatching, distort algorithmic routing decisions or cause specialized models to default to global fallbacks. Manually isolating silent failures between initial candidate generation and post-processing filters using specific engineering tools represents an advanced, industrial troubleshooting methodology
--	--	--	--

		geometric deduplication failed to merge noisy beam candidates, compiling the coordinates using Shapely for the engineering team. 5. Column Signal & Diff Generation: Isolated failures in the Column Geometric Baseline to determine if missed columns resulted from unrecognized non-standard markers or over-aggressive gating. Finally, I ran <code>diff_candidate_runs.py</code> to confirm whether missed labels were dropping out during initial generation or being improperly filtered during post-processing alignment.	completely absent from my traditional curriculum. • If not related in the school, discuss the importance of the learning that it should be taught in the school This type of end-to-end pipeline analysis is highly important and should be integrated into university curricula because real-world machine learning systems are rarely simple, single-step operations. Students frequently graduate knowing how to calculate accuracy metrics on static data, but they lack the skills to diagnose performance degradation within an intricate, multi-stage production environment. Teaching students how to systematically identify where data drops out, whether in the ingestion, routing, or post-processing deduplication phase, is essential for building resilient engineering systems. Introducing workflows that account for messy, real-world data noise and showing how it impacts specialized sub-models would bridge a massive gap, producing data scientists capable of handling complex enterprise architectures.
5/22/2026	8hrs	What did you do that day? Authored a comprehensive technical analysis report diagnosing hurdles in the detection pipeline, executed diagnostic scripts to trace feature space inflation, and attended a DSM to realign our dataset prioritization and propose architectural adjustments. Quality of Experience:	• What did you learn? During this shift, I learned how to translate complex data pipeline errors into strategic architectural proposals for machine learning optimization. Running diagnostic scripts showed me how dense engineering layouts overwhelm a geometric candidate generator, inflating the feature space and degrading downstream LightGBM model precision. I also learned how aggressive gating filters out

		Highly strategic and engineering-focused. Transitioning from identifying errors to actively proposing pipeline architecture changes, like noise suppression and spatial matrices, provided a true full-stack machine learning experience. Steps or the Flow: 1. Technical Hurdles Analysis: Documented recurring source-data issues across framing-plan PDFs. I systematically categorized errors into False Positives caused by heavy background noise or existing markups, Missed Members, and incomplete beam annotations. 2. Diagnostic Execution: Utilized <code>diff_candidate_runs.py</code> and <code>compare_manual_counts.py</code> to quantify the impact of these errors. The diagnostics revealed that dense layouts were overwhelming the Geometric Candidate Generator. This inflated the feature space and degraded the precision of the downstream LightGBM specialist models. Additionally, I isolated poor column detection to non-standard geometric markers being aggressively gated out before ML scoring could occur. 3. DSM & Strategic Realignment: Attended the Daily Stand-up Meeting to discuss dataset prioritization. The team agreed to shift focus to company-provided structural PDFs to better reflect realistic client workflows and production conditions. 4. Pipeline Adjustment Proposals: Outlined technical interventions for the next phase. This included calibrating the PDF Cluster Router to ensure correct routing to the C1, C2, and C3	non-standard markers, causing missed column detections. This investigation taught me how to conceptualize system interventions, such as prototyping a YOLO or ONNX masking layer to filter background noise and designing a spatial proximity matrix to link text labels to elements. Finally, I learned how agile teams use daily stand-ups to dynamically realign dataset priorities based on production workflows. • Describe if the task was already taught in school or something new. While my university coursework introduced general machine learning theory and basic error metrics, the task of diagnosing multi-stage architectural bottlenecks and proposing system-level modifications is completely new. In a classroom, assignments rarely encounter feature space inflation caused by raw file ingestion complexities, nor do they require debugging why early gating stages aggressively reject valid data. Proposing industrial solutions, like integrating an upstream YOLO model on ONNX to mask noise or designing spatial proximity matrices for label alignment, represents a structural engineering workflow absent from academia. Additionally, collaboratively pivoting dataset strategy during a daily stand-up to meet production deadlines is a corporate reality not simulated in school. • If not related in the school, discuss the importance of the
--	--	---	--

		models, prototyping a YOLO/ONNX masking step to exclude markups before candidate generation, and proposing an updated spatial proximity matrix to forcefully link textual labels to unannotated beams.	learning that it should be taught in the school This systems-level troubleshooting is incredibly important and should be taught in universities because real-world machine learning requires deep architectural thinking rather than just basic hyperparameter tuning. Academic courses often focus on optimizing isolated models using static data, leaving students unprepared for scenarios where an upstream processing layer inflates the feature space and cripples downstream classifiers. Training students to critically evaluate a failing pipeline, pinpoint architectural bottlenecks, and propose multi-layered algorithmic interventions builds essential engineering foresight. Furthermore, learning to dynamically realign project priorities during agile ceremonies based on operational constraints bridges the gap between theory and industry execution, preparing graduates to confidently drive technical decisions.
5/25/2026	8 hrs	What did you do that day? Participated in a column-annotation coordination meeting, reviewed a 108-page structural PDF for column detection outputs, proposed advanced geometric heuristics to fix specific failure modes, and attended a DSM evaluating the combined Geometric + ML approach for beam detection. Quality of Experience: Highly technical and solution-oriented. Transitioning from identifying column detection failures to designing specific geometric heuristics and intersection	What did you learn? During this shift, I learned how to combine advanced geometric heuristics with machine learning to resolve specialized computer vision failures in object detection pipelines. By auditing a 108-page structural PDF, I learned to isolate distinct column-detection failure modes, including hollow architectural false positives, missing implicit grid labels, and complex steel profile omissions. I learned how to design targeted programmatic solutions using the Shapely library to filter out non-column elements via fill ratios

		algorithms demonstrated a strong progression into advanced computer vision engineering. Steps or the Flow: 1. Column Detection Validation: Reviewed a 108-page structural PDF, specifically analyzing the Column Geometric Baseline. I identified False Positives (hollow architectural features), Missed Labels (implied grid intersections), and Missed Members (complex steel profiles). 2. Heuristic Pipeline Proposals: Designed technical interventions to address the identified failures: Proposed using Shapely to calculate fill ratios and perimeter-to-area metrics to filter out non-column squares (like ductwork) to reduce False Positives. Proposed integrating the YOLO Symbol Detector to extract grid axes, using PyMuPDF to calculate topological intersections to infer column labels that lack text. Recommended expanding feature engineering to recognize multi-line profiles like I-beams, preventing them from being gated out. 3. DSM & Evaluation Review: Attended the 8 PM DSM to review evaluation metrics for beam detection. We analyzed the performance of the geometric baseline, the pure ML model, and the combined Geometric + ML approach. We discussed scenarios where the combined approach improved overall performance, as well as edge cases where both failed,	and perimeter-to-area metrics, as well as leveraging PyMuPDF topological intersection tracking alongside a YOLO Symbol Detector to infer missing labels. Finally, through our daily stand-up meeting, I learned how to quantitatively evaluate hybrid systems by analyzing how a combined Geometric and machine learning approach optimizes beam detection across complex client requirements compared to standalone models. • Describe if the task was already taught in school or something new. While my university coursework covered basic computer vision concepts, standard machine learning theory, and primitive geometry, this process of designing custom algorithmic heuristics to repair a production pipeline is entirely new to me. In an academic setting, object detection assignments are typically bounded by traditional bounding-box models trained on standard datasets, completely bypassing the need to write auxiliary geometric logic. Developing custom spatial filters using Shapely to clean noisy candidate generation layers or calculating vector intersections with PyMuPDF to infer missing data is an industrial pipeline-engineering workflow absent from school. Furthermore, evaluating the performance tradeoffs of a hybrid Geometric and machine learning system against strict enterprise deployment standards represents a sophisticated level of systems analysis that goes far beyond classroom exercises.
--	--	---	---

		ensuring alignment with specific client requirements for Project SteelSnap.	• If not related in the school, discuss the importance of the learning that it should be taught in the school This advanced approach to system debugging is highly important and should be integrated into university curricula because real-world artificial intelligence deployment frequently relies on hybrid architectures rather than purely raw model outputs. Academic programs often graduate students with a narrow focus on building larger models, leaving them underprepared for production environments where hardware, data noise, or edge cases require lightweight geometric preprocessing or post-processing logic. Teaching students how to design rules-based heuristics to complement machine learning models trains them to think like practical engineers who can systematically eliminate false positives and handle unstructured data constraints. Incorporating hybrid pipeline design and cross-layer debugging into the classroom would give students the vital architectural foresight needed to build scalable, high-performance commercial applications.
5/26/2026	8hrs	What did you do that day? Attended a coordination meeting for annotation distribution, validated four structural PDF plan sets, and authored a detailed diagnostic analysis on how source-data quality issues trigger specific pipeline failure modes. Quality of Experience:	• What did you learn? During this shift, I learned how raw source-data quality issues trigger compounding classification errors across distinct layers of an enterprise machine learning pipeline. By validating four structural PDF plan sets, I learned how overlapping raster text and irregular architectural hatching degrade the Geometric

	Highly analytical and system-focused. Connecting visual drawing artifacts directly to backend classification errors like "Cluster Bleed" provided an advanced understanding of enterprise data pipelines. Steps or the Flow: 1. Coordination & Alignment: Participated in a quick alignment meeting to receive new annotation assignments and clarify structural drawing interpretation guidelines with the team. 2. Validation & Annotation Review: Completed a thorough review of four distinct structural PDF plan sets. I systematically documented source-data quality issues, paying specific attention to noisy linework, drawing artifacts, and inconsistent geometric elements. 3. Artifact Interference Analysis: Analyzed how heavy drawing artifacts, such as irregular architectural hatching and raster text overlapping vector lines, degraded the Geometric Signal-to-Noise Ratio (GSRN). I documented how this exact noise caused the Geometric Candidate Generator to over-propose False Positive beam candidates, inflating the feature space and stressing the downstream LightGBM evaluators. 4. Cluster Bleed Diagnosis: Investigated routing anomalies within the PDF Cluster Router. I tracked instances of "Cluster Bleed" where variations in line weights on a single page confused the classifier. I reported that this caused drawings meant for the C1, C2, or C3 (clean/bold) pipelines to be misclassified and incorrectly routed	Signal-to-Noise Ratio (GSRN), forcing the candidate generator to over-propose false positives and inflate the downstream feature space. I also gained deep technical insight into routing anomalies, specifically diagnosing "Cluster Bleed" where line-weight variations confuse the PDF Cluster Router, causing bold-line drawings to misroute into thin-line specialist or global fallback models. Finally, I learned how dense, noisy linework can mimic valid structural nodes, deceiving post-processing geometric deduplication algorithms into incorrectly merging false positives into valid beam coordinates. ● Describe if the task was already taught in school or something new. While my university data science classes introduced general classification metrics and basic data processing, this advanced pipeline triage and structural failure analysis is completely new to me. In an academic setting, datasets are uniformly clean, meaning I never encountered physical file artifacts like line-weight fluctuations or raster-vector overlap that could disrupt an entire system. School never exposed me to architectural concepts like "Cluster Bleed," nor did it require me to track how an upstream routing misclassification causes a domino effect that degrades downstream thresholding and deduplication. Analyzing how intricate visual noise corrupts spatial engineering logic across independent, version-controlled pipeline modules represents a
--	--	--

		to the C0/C4 (thin-line) specialists or the global fallback model, leading to sub-optimal thresholding and missed detections. 5. Failure Review: Analyzed the post-processing stages, noting that dense, noisy linework frequently mimicked valid structural nodes like beam-column connections. I reported that this caused the per-source threshold geometric deduplication stage to incorrectly merge overlapping false positives into valid beam coordinates, ultimately skewing the final detection output boundaries.	highly specialized industrial debugging workflow absent from traditional coursework. • If not related in the school, discuss the importance of the learning that it should be taught in the school This systems-level troubleshooting is highly important and should be taught in universities because real-world machine learning deployment is rarely a clean, single-step operation. Students frequently graduate knowing how to maximize mathematical accuracy on pristine datasets, but they lack the practical engineering skills to diagnose what happens when physical data noise skews a multi-stage model architecture. Training students to systematically identify where data anomalies degrade specific pipeline components whether during candidate generation, router allocation, or geometric post-processing builds essential architectural resilience. Introducing concepts like data-driven routing and multi-layer error taxonomy into the classroom would bridge a massive gap, producing data scientists capable of optimizing scalable, production-grade AI systems.
5/27/2026	8hrs	What did you do that day? Executed targeted column detection testing, actively filtered the PDF dataset to remove noisy source files, wrapped the detection modules in Docker containers for deployment consistency, and attended the 8:00 PM	• What did you learn? During this shift, I learned the practical intersection of data operations and MLOps deployment strategies required to meet tight client presentation deadlines. I learned how to execute strategic data sanitization by identifying and filtering out low-quality, noisy files that artificially deflate model metrics

	DSM to align on a two-week client presentation sprint. Quality of Experience: Highly strategic and operations-focused. Balancing data sanitization with MLOps deployment tasks provided a comprehensive look at how a team prepares an engine for a major client milestone. Steps or the Flow: 1. Data Sanitization & Filtering: To protect the model's training and evaluation baselines, I conducted rigorous PDF filtering. I actively removed noisy, low-quality, and unsuitable plan sets that were artificially deflating our F1 scores. 2. Column Detection Improvement: Transitioned to column-specific tasks. I manually annotated complex column members and analyzed existing detection outputs, documenting specific edge cases involving Missed Members and False Positives to feed back into the geometric baseline. 3. Model Module Containerization (Docker): To ensure our testing environments remained perfectly synchronized during the two-week sprint, I assisted in wrapping the core detection modules into Docker containers. I configured Dockerfiles to encapsulate the Python environment, ensuring libraries like PyMuPDF, LightGBM, and ONNX Runtime were locked to specific versions. This eliminated "it works on my machine" errors and streamlined the deployment of the benchmark.py scripts across the engineering team.	like F1 scores. On the infrastructure side, I learned how to containerize machine learning modules using Docker, configuring Dockerfiles to encapsulate Python environments and lock critical libraries like PyMuPDF, LightGBM, and ONNX Runtime to eliminate environment drift across the engineering team. Finally, I learned the tactical logic behind geometric sequencing in computer vision pipelines, understanding why column detection must be optimized first to serve as the spatial anchor for subsequent beam-to-column segmentation. • Describe if the task was already taught in school or something new. This experience introduced several advanced engineering and operations workflows that are entirely new compared to my academic background. While university courses teach the theoretical definitions of F1 scores and containerization concepts, they rarely require students to actively prune real-world datasets to sanitize performance baselines. School assignments typically provide fixed, unalterable datasets, completely bypassing the industry practice of data filtering. Furthermore, writing production-grade Dockerfiles to bundle complex, multi-model dependencies like ONNX Runtime and LightGBM for a live engineering team is a mature MLOps practice that extends far beyond building simple, isolated scripts for a classroom grade.
--	--	--

		4. DSM & Strategic Realignment: Attended the evening DSM where the team established a strict priority shift. We determined that column detection must be optimized first, as accurate column identification serves as the geometric anchor required for proper beam-to-column segmentation. This strategy is strictly geared toward delivering measurable F1 improvements to the clients within a two-week window.	• If not related in the school, discuss the importance of the learning that it should be taught in the school This combination of data sanitization, containerization, and dependency locking is highly important and should be a staple in university tech curricula. Traditional academic programs heavily focus on modeling, leaving graduates underprepared for the operational realities of deployment where environment mismatches and messy data stall production. Teaching students how to use Docker to ensure deployment consistency across different systems is a critical skill for any modern software engineer or data scientist. Additionally, educating students on how to prioritize architectural components based on spatial dependencies such as anchoring segmentations around a specific node builds the practical system-design thinking required to deliver successful commercial AI solutions under tight constraints.
5/28/2026	8hrs	What did you do that day? The intern team was categorized into specialized groups to address major pipeline bottlenecks identified by the tech lead. I was assigned to investigate the "High-Vector PDF" processing failure, researching Python and TypeScript libraries to develop layer flattening and raster-conversion preprocessing techniques. Quality of Experience: Highly research-intensive and optimization-focused. Shifting from data validation to core computational	• What did you learn? During this shift, I learned how to diagnose and mitigate catastrophic memory bottlenecks caused by processing high-density vector PDFs. I learned that excessive vector paths from raw CAD exports can cause a parsing engine to exceed memory limits and hang during file ingestion. To resolve this, I learned to design preprocessing strategies like dynamic layer flattening, vector simplification, and conditional raster-conversion fallbacks that act as a circuit breaker by rendering heavy vector pages into high-DPI images when

	efficiency provided a deep dive into memory management, rendering logic, and building resilient data pipelines. Steps or the Flow: 1. Task Categorization: After the tech lead collated our recent analysis reports, we were divided to tackle specific system weaknesses. I took ownership of the high-vector density issue, where the pdf_engine.py parsing tool hangs or fails completely due to the sheer complexity of vector layers. 2. Bottleneck Profiling: I reviewed the specific structural PDFs that triggered the failure. I analyzed the computational load, noting that excessive vector paths like dense architectural cross-hatching or raw CAD exports were causing the system to exceed memory limits before any geometric candidates could even be generated. 3. Preprocessing Strategy Prototyping: I conducted a technical assessment of several mitigation techniques. I outlined logic for dynamic layer flattening to compress the PDF structure and vector simplification to merge redundant lines before processing. 4. Raster-Conversion Fallback: I proposed a conditional raster-based conversion strategy. If a script detects that a page's vector object count exceeds a safe threshold, the system would automatically render the page as a high-DPI raster image. This acts as a circuit breaker, bypassing the vector-parsing crash and sending a clean image directly to the downstream computer vision models.	object counts pass a safe threshold. Finally, I learned to evaluate full-stack performance optimization by exploring python modules like PyMuPDF and Ghostscript for backend handling, alongside frontend TypeScript libraries like pdf.js to offload heavy rendering tasks before data hits the inference engine. ● Describe if the task was already taught in school or something new. This research-intensive optimization task is entirely new compared to my academic coursework. While university projects teach basic file reading and model training, they use clean, static datasets that never stress system memory or cause parsing engines to crash. School never exposed me to the computational realities of processing industrial-grade vector graphics, nor did it teach me how to profile backend memory leaks or design multi-language preprocessing architectures using Python and TypeScript to handle unparsed data. ● If not related in the school, discuss the importance of the learning that it should be taught in the school This systems-level memory optimization is critical and should be integrated into university curricula because production-level engineering regularly faces unstructured, uncurated data that can crash an application before any AI model even runs. Teaching students how to profile
--	---	--

		5. Library Investigation: To implement these fixes, I researched robust libraries. I evaluated advanced Python modules (like specialized PyMuPDF functions and Ghostscript wrappers) for backend vector flattening. I also investigated TypeScript libraries (such as pdf.js) to explore if we could offload some of the heavy rasterization tasks to the frontend client before the data hits our inference engine.	performance bottlenecks, handle heavy file rendering, and build architectural circuit breakers prepares them for real-world deployment challenges. Shifting academic focus from theoretical modeling toward core computational efficiency and data-ingestion resilience would create graduates capable of building highly scalable, commercial data pipelines.
5/29/2022 6	8hrs	What did you do that day? Executed precision ground-truth annotation for beam members to enhance detection accuracy. The focus was on standardizing geometric boundaries, targeting noisy edge cases, and linking semantic labels before exporting the data for re-benchmarking. Quality of Experience: Highly detail-oriented and foundational. Returning to manual annotation with a strict engineering mindset reinforced how crucial high-quality, standardized ground truth is for resolving complex mathematical evaluation errors. Steps or the Flow: 1. Annotation Protocol Standardization: Replaced loose bounding-box approximations with precise line-string geometries that map exactly to the centerlines of the structural members. This protocol ensures that the compare_manual_counts.py script and the Shapely spatial proximity matrix can accurately calculate Intersection	• What did you learn? During this shift, I learned how precision data standardization directly resolves mathematical evaluation errors within a machine learning pipeline. By replacing loose bounding boxes with precise line-string geometries mapping exactly to member centerlines, I learned how to optimize Intersection over Union (IoU) calculations for automated benchmark runners. I also learned how targeted data curation isolates valid features from noisy architectural hatching, preventing LightGBM models from learning false geometric patterns. Finally, I learned how semantic label linking creates the exact training targets needed to calibrate post-processing alignment scripts and fix missed label errors. • Describe if the task was already taught in school or something new. While my university courses taught the general definitions of IoU, data labeling, and machine learning baselines, the strict, programmatic geometry formatting I executed today is completely new. Academic assignments always provide

		over Union (IoU) during automated benchmarking. 2. Edge Case Targeting: Focused manual annotation efforts on historically problematic drawing zones, specifically sheets suffering from heavy background noise or dense architectural hatching. Explicitly separating valid beam vectors from stray linework prevents the LightGBM candidate scoring models from learning false geometric patterns. 3. Semantic Label Linking: Addressed the Missed Labels issue by manually mapping the relationships between beam annotations and their text callouts. This provided the exact matching targets required to train and calibrate the post-processing alignment stage. 4. JSON Export & Re-Benchmarking: Exported the newly annotated plan sets into the standardized JSON schema required by the detection engine. Finally, I scheduled these upgraded files to be processed through <code>run_sharded_benchmark.py</code> to establish a new evaluation baseline.	pre-labeled datasets or expect basic bounding boxes, completely bypassing the engineering rigor needed to design custom line-string geometries or manually map semantic relationships between text callouts and spatial coordinates to train complex post-processing layers. • If not related in the school, discuss the importance of the learning that it should be taught in the school This meticulous approach to data engineering is incredibly important and belongs in the classroom because high-quality ground truth is the absolute foundation of any successful production model. Universities often graduate students who only know how to tune hyperparameters, leaving them unprepared for industry roles where fixing a model requires restructuring the underlying dataset geometry. Teaching students how to build precise, standardized annotations and design semantic data links would bridge a major gap, training them to treat data curation as a core engineering discipline.
6/1/2026	8hrs	What did you do that day? Conducted annotation tests and automated benchmarking. Exported validated PDF annotations into JSON format, performed Exploratory Data Analysis (EDA), and calculated standard deviations to compare detection outputs against the ground truth. Concluded the day with analysis reporting and a Project SteelSnap meeting.	• What did you learn? I learned how to apply descriptive statistics to quantify model drift and annotation consistency within a machine learning pipeline. By calculating the standard deviations of geometric coordinates and confidence scores using Pandas, I learned how to mathematically measure the variance between manual ground-truth baselines and automated predictions. This

		Quality of Experience: Highly analytical and statistically driven. Applying variance analysis to error metrics provided a quantifiable way to measure model drift and annotation consistency. Steps or the Flow: 1. Annotation & Benchmarking: Completed manual annotation tests on specific structural use-case PDFs. I then ran the benchmark.py workflow to evaluate the model's current spatial accuracy. 2. Data Export & Structuring: Exported my validated, correct annotated versions into the required structured JSON schema. 3. Statistical EDA: Loaded the JSON candidate files using Pandas to perform Exploratory Data Analysis. I specifically calculated the standard deviations of the geometric coordinates and confidence scores to quantify the variance between the machine's predictions and my manual ground-truth baselines. 4. Reporting & Alignment: Compiled these statistical findings into a formal analysis report. I presented these insights during the project meeting to help the engineering team identify systematic biases in the geometric candidate generation phase.	statistical approach allowed me to translate qualitative visual errors into quantitative metrics, providing the engineering team with clear indicators of systematic bias occurring during the geometric candidate generation phase. • Describe if the task was already taught in school or something new. This workflow combined familiar academic tools with entirely new industry applications. While my university data science classes thoroughly taught me how to calculate standard deviations and perform exploratory data analysis using Pandas, applying these statistical concepts to multi-dimensional geometric coordinates within an active computer vision pipeline is completely new. School projects typically leverage these statistics on simple tabular datasets rather than utilizing them to track spatial model drift against manual ground-truth schemas. • If not related in the school, discuss the importance of the learning that it should be taught in the school This technique is highly important and should be taught in universities because it shows students how to use basic statistics to solve real-world machine learning issues like model drift. In academia, model performance is often treated as a static score, but in production, engineers must constantly monitor and quantify prediction variance. Teaching students how to build
--	--	---	---

			statistical feedback loops using descriptive metrics would better prepare them to diagnose systematic biases in commercial AI infrastructure.
6/2/2026	8hrs	What did you do that day? Attended a use-case synchronization meeting, calibrated the machine learning models using recent JSON data, conducted targeted annotation and benchmarking, and cleared a company-wide bottleneck by reviewing 10 Pull Requests (PRs) across other projects using Codex CLI. Quality of Experience: Highly dynamic and cross-functional. Stepping out of the core SteelSnap pipeline to assist with a broader engineering bottleneck demonstrated the fast-paced, collaborative nature of a live production environment. Steps or the Flow: 1. Use-Case Alignment: Started with a short meeting to define the specific structural drawing use cases that required immediate optimization. 2. ML Model Adjustments: Utilized the structured JSON data and standard deviation insights gathered the previous day to make targeted adjustments to the machine learning models. This involved fine-tuning confidence thresholds for the LightGBM specialists to better handle the newly defined use cases. 3. Annotation & Benchmarking: Annotated a fresh batch of structural PDFs specifically tailored to these new	• What did you learn? I learned how to use statistical variance insights to calibrate confidence thresholds for specialized LightGBM models to handle targeted structural use cases. Additionally, I gained practical experience in cross-project dev operations and team triage. By stepping out of my core pipeline tasks, I learned how to use the Linear app and Codex CLI to efficiently navigate, analyze, and review ten pending pull requests across separate corporate repositories, ensuring code integration standards were maintained during a team bottleneck. • Describe if the task was already taught in school or something new. While school provided the foundational theory behind model thresholds and basic version control, the cross-functional engineering tasks I executed are entirely new. Academic settings rarely require students to calibrate specialized ensemble sub-models based on real-time statistical feedback. Furthermore, utilizing advanced AI-assisted tools like Codex CLI to conduct rapid, multi-repository pull request reviews to clear organizational bottlenecks is a fast-paced DevOps workflow completely absent from traditional coursework.

		use cases. I then executed the benchmark.py and run_sharded_benchmark.py scripts to evaluate how the threshold adjustments impacted the engine's precision and recall. 4. Cross-Project PR Review: The engineering team experienced a ticket bottleneck, so I was reassigned to help clear the queue. Using the Linear app to track assignments, I navigated to GitHub and utilized the Codex CLI tool to efficiently analyze and review 10 PRs across other company projects, ensuring code quality and integration standards were met before approval.	• If not related in the school, discuss the importance of the learning that it should be taught in the school Exposing students to pull request reviews and multi-repository management using modern tools like Codex CLI is incredibly vital for modern software engineering. Universities heavily focus on writing isolated code, leaving graduates unprepared for the collaborative reality of maintaining code quality across large, interconnected enterprise repositories. Integrating structured peer code reviews and agile issue tracking into curricula would drastically shorten the onboarding time for junior data scientists and engineers entering the workforce.
6/3/2026	8hrs	What did you do that day? Attended a late-evening project meeting to align on pipeline priorities, conducted dual-element annotations for both columns and beams, executed automated Python benchmarking, and continued assisting with cross-project Pull Request (PR) reviews. Quality of Experience: Highly focused and deadline-driven. Managing competing project discussions while executing critical path tasks for an impending deadline provided a realistic view of production triage. Steps or the Flow: 1. Strategic Synchronization Meeting (8 PM - 10 PM): Attended a project alignment meeting. The team	• What did you learn? I learned how to design and evaluate complex relational data mappings by executing dual-element annotations for overlapping structural components. By manually mapping both beam lines and column members within the same document, I learned how to format relational geometric dependencies into JSON schemas. This allowed me to analyze how the simultaneous presence of interconnected physical elements impacts downstream LightGBM candidate scoring, specifically evaluating how spatial relationships influence false-positive and missed-member rates under strict production constraints. • Describe if the task was already taught in school or something new.

		discussed parallel developments for the "Diamond Players" project, but leadership mandated that all intern efforts remain strictly focused on Project SteelSnap due to the tight two-week client deadline. 2. Dual-Element Annotation & Export: Shifted from isolated testing to dual-element annotation. I manually annotated both column members and beam lines within the same structural PDFs to establish their geometric relationships. I then exported these complex relational mappings into structured JSON files. 3. Python Benchmarking & Analysis: Ran the updated JSON files through the benchmark.py and run_sharded_benchmark.py scripts. I compiled an analysis report evaluating how the combined presence of columns and beams affected the False Positive and Missed Member rates in the downstream LightGBM models. 4. Cross-Project PR Review: To prevent engineering bottlenecks during the deadline push, I utilized the Linear app and Codex CLI tools to review pending PRs, ensuring code quality and maintaining the momentum of other company projects.	This task is entirely new compared to my academic training. In a university setting, data labeling assignments are typically restricted to single-class object detection, completely bypassing relational mapping. School never exposed me to the complexities of formatting interdependent geometric structures into a single hierarchical JSON schema, nor did it teach me to evaluate how the performance metrics of one model class (columns) dynamically influence the error rates of another (beams) in a production pipeline. ● If not related in the school, discuss the importance of the learning that it should be taught in the school Teaching relational data annotation and multi-class pipeline evaluation is highly critical because commercial AI applications rarely process data in isolation. Real-world systems depend on interconnected features, such as structural nodes where beams anchor to columns. If universities introduce students to multi-layered data modeling and teach them to analyze how compound features affect downstream error metrics, graduates will possess the systemic thinking required to build resilient, production-grade computer vision applications.
6/4/2026	8hrs	What did you do that day? Participated in a strategic pivot where active machine learning development was paused to adopt a strictly data-centric approach. I annotated 20 structural PDFs specifically targeting curved beam geometries, converted	● What did you learn? I learned the practical execution of a data-centric machine learning pivot, where active model training is paused to systematically address underlying data quality failures. By isolating a critical failure mode

		them to JSON, and performed a standard deviation analysis to compare outputs against the ground truth. Quality of Experience: Highly grounding and insightful. Experiencing a hard pause on model development to fix the underlying data reinforced the philosophy that a machine learning model is only as intelligent as the annotations it learns from. Steps or the Flow: 1. Strategic Pivot: Leadership mandated a pause on ML model tuning to focus entirely on data quality and error analysis. We identified a critical failure mode: the pipeline was systematically failing to detect beams with curved shapes or complex arcs. 2. Targeted Annotation: I manually annotated a batch of 20 structural PDFs. I focused explicitly on accurately mapping the geometric boundaries of curved beam members, ensuring pdf_engine.py had a precise reference for non-linear shapes that typically confuse the candidate generator. 3. JSON Export & Benchmarking: Exported the newly annotated plan sets into the required JSON schema. I then ran evaluation workflows like compare_manual_counts.py to establish a baseline for these specific edge cases. 4. Statistical Analysis: I used Pandas to calculate the standard deviation between the machine's geometric predictions and my newly established manual ground truth. This quantified exactly how much the LightGBM	involving non-linear shapes, I learned how to curate and annotate targeted datasets for curved beam geometries to provide a precise reference for a geometric candidate generator. Additionally, I learned to use standard deviation analysis to calculate exactly how much a LightGBM model drifts mathematically when encountering complex, non-linear vector arrays. • Describe if the task was already taught in school or something new. The data-centric philosophy practiced during this shift is completely new. Academic curricula are heavily model-centric, instructing students to constantly tweak hyperparameters, adjust architectures, or swap algorithms to fix accuracy drops on unchanging, pre-cleaned datasets. School never prepares students to execute a hard pause on development to manually re-engineer the dataset geometry or resolve feature extraction errors by targeting specific non-linear edge cases within a raw file parser. • If not related in the school, discuss the importance of the learning that it should be taught in the school This learning is incredibly important because industry engineers solve performance bottlenecks by refining data quality rather than continuously rewriting model code. Shifting the academic narrative toward data-centric engineering would teach students to treat data curation as a rigorous, iterative
--	--	--	---

		models were drifting when presented with curved vectors. 5. Reporting: Compiled the EDA and statistical variance findings into a formal analysis report and submitted it to my immediate supervisor to guide the next phase of data collection.	discipline. Training students to identify systematic geometric failures and actively expand datasets with precise edge-case annotations would yield data scientists who can effectively optimize real-world models.
6/5/2026	8hrs	What did you do that day? Updated my local development environment with new repository modules, validated JSON coordinate accuracy for recent annotations, and analyzed the resulting trade-off in model performance. I concluded the day with a late-evening alignment meeting regarding client updates and use cases. Quality of Experience: Highly analytical and reflective. Experiencing the classic machine learning trade-off, where fixing one failure mode introduces another, provided a mature perspective on model calibration. Steps or the Flow: Environment Sync & Documentation: Pulled the latest technical updates from the private GitHub repository. I rebuilt my local working environment to include the new pipeline modules and reviewed the updated technical documentation to understand the backend changes. JSON Coordinate Validation: Conducted strict annotation checking and validation. I verified the coordinate accuracy of the exported JSON files, ensuring the newly mapped curved	• What did you learn? I learned how to navigate and evaluate systemic machine learning trade-offs, where resolving one specific failure mode introduces a new vector for errors. While correcting the model's curved-beam detection architecture, I observed how expanding the geometric candidate generator caused it to over-extract non-structural curved architectural markups, leading to a spike in false positives. I learned how to document these use-case trade-offs and analyze how relaxed structural logic stresses downstream LightGBM evaluators, requiring precise per-source thresholding calibration to filter out the resulting noise. • Describe if the task was already taught in school or something new. Diagnosing and managing live model regressions and post-processing trade-offs is entirely new compared to my academic coursework. In a traditional university setting, assignments are designed so that clean data improvements yield linear progress across all metrics. School rarely exposes students to the frustrating reality where fixing a precision error immediately causes

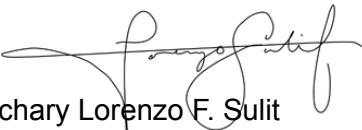
		beam geometries aligned perfectly with the strict spatial schema required by the compare_manual_counts.py script. 3. Benchmarking & Trade-off Analysis: Ran the updated JSON files through the benchmarking workflows. I discovered that while our recent data-centric pivot successfully improved the detection of curved beams (reducing Missed Members), the model now produced a significant volume of False Positives. The Geometric Candidate Generator was now over-extracting non-structural curved architectural markups. 4. Use-Case Reporting: Compiled these findings into a formal use-case report, detailing exactly how the expanded geometric logic stressed the downstream LightGBM evaluators. 5. Project Meeting (8 PM - 10 PM): Attended the late-evening team meeting to discuss these client updates, review the specific use-case trade-offs, and brainstorm strategies to calibrate the per-source thresholding to filter out the new false positives.	a drop in recall, nor does it teach students how to adjust multi-layered, per-source post-processing thresholds to balance these competing model behaviors. • If not related in the school, discuss the importance of the learning that it should be taught in the school This experience highlights a fundamental reality of applied machine learning that belongs in the classroom. Universities must teach students that production AI is a game of balancing trade-offs, not just chasing a perfect, isolated metric. Understanding how a change in the ingestion layer impacts downstream classification and post-processing filters builds crucial systems-level foresight. Teaching students to analyze and resolve these cascading dependencies would prepare them to deploy and maintain complex, balanced models in enterprise environments.
6/8/2026	8hrs	What did you do that day? Attended a pivotal strategy meeting introducing a new Agentic AI architecture. I collated all my previous technical analysis reports to feed this new agent's knowledge base and finalized my last batch of 50 structural PDF annotations. Quality of Experience: Highly evolutionary and satisfying. Seeing weeks of manual error analysis	• What did you learn? I learned how qualitative documentation and empirical error logs are transformed into structured knowledge bases for retrieval-augmented generation (RAG) within an Agentic AI architecture. By securing a final ground-truth baseline of 50 structural PDFs and converting my past technical reports into Markdown files, I learned how raw data-centric analysis directly feeds an agent's context window. During

	and reporting being directly utilized to provide context for a cutting-edge Agentic LLM framework felt like the ultimate culmination of the data-centric sprint. Steps or the Flow: 1. Final Annotation Batch: Executed the final manual annotation, validation, and JSON export for a batch of 50 structural PDFs, securing the geometric ground-truth baseline for the current evaluation cycle. 2. Report Collation & Knowledge Extraction: Aggregated all my previous detection testing reports, false-positive analyses, and use-case edge case documentation. I organized these findings so they could be effectively converted into Markdown (.md) files. 3. Agentic Pipeline Strategy Meeting (8 PM - 10 PM): Attended a major architectural alignment meeting led by the AI Engineering Tech Lead. To match competitor capabilities, the team introduced an "Agentic Approach" designed to run alongside the PDF Cluster Router. 4. Context Provisioning: Handed over my collated .md reports to the tech lead. The new agent is engineered to dynamically ingest these files and trained datasets to "learn" specific structural drawing use cases, allowing it to reason through complex edge cases that the rigid geometric baselines and LightGBM models traditionally fail on.	our alignment meeting, I observed how an enterprise system integrates a reasoning LLM agent alongside traditional infrastructure like a PDF Cluster Router and LightGBM models allowing the agent to dynamically process the ingested context to reason through complex layout edge cases that rigid algorithmic baselines traditionally fail to resolve. • Describe if the task was already taught in school or something new. This experience is entirely new compared to my academic coursework. While universities introduce the basic theoretical definitions of large language models or agentic concepts, they do not teach the practical systems engineering required to deploy them in a hybrid production environment. School projects typically rely on closed, tabular datasets or fine-tuning isolated models, completely bypassing the industrial workflow of using Markdown-formatted empirical error logs as a live, contextual knowledge base to augment a multi-stage computer vision and reasoning pipeline. • If not related in the school, discuss the importance of the learning that it should be taught in the school This integration is highly critical and belongs in university classrooms because the tech industry is rapidly shifting away from monolithic models toward hybrid, agentic architectures. Data science students need to learn that their
--	---	--

			technical documentation and error taxonomies are not just static homework assignments, but valuable unstructured data assets that can optimize an AI system's contextual reasoning. Teaching students how to structure data for RAG and orchestrate LLM agents alongside classical machine learning models would prepare them to design the sophisticated, multi-layered systems expected in commercial tech roles.
6/9/2026	8hrs	What did you do that day? Continued context provisioning for the new Agentic AI architecture while ML tweaking remains paused. Conducted PR reviews using Codex CLI, specifically addressing a UI coordinate bug, and executed detection testing on the newly trained agent, identifying a vulnerability to unfamiliar text fonts. Quality of Experience: Highly diagnostic and interdisciplinary. Balancing front-end UX debugging with back-end Agentic AI evaluation provided a comprehensive, full-stack perspective on the product's deployment. Steps or the Flow: 1. Context Provisioning & Analysis: Continued the data-centric pivot, actively curating and formatting context for the Agentic AI while traditional ML hyperparameter tuning remained paused. 2. Cross-Project PR Review (Codex CLI): Utilized the Codex CLI to review incoming Pull Requests. I specifically tackled a front-end UI ticket where	• What did you learn? I learned how to evaluate an agentic AI system for typographic vulnerabilities and diagnose how unstandardized source files can disrupt downstream contextual reasoning layers. Testing the newly deployed agent revealed that variations in text fonts across external PDF distributors corrupted the system's text extraction, causing the agent to misread critical labels despite having the proper Markdown context. Additionally, I advanced my full-stack debugging skills by using the Codex CLI during pull request reviews to isolate a front-end UI alignment bug, correcting coordinate offset logic so that copy-pasted structural assets center precisely on a user's cursor rather than snapping left. • Describe if the task was already taught in school or something new. While my university courses provided the mathematical foundations for coordinate systems and text parsing, this interdisciplinary testing and

		"copy-pasted" structural members on the staging website were snapping to the left instead of centering precisely on the user's arrowhead/cursor. I reviewed the coordinate offset logic, validated the fix, and submitted the approval. 3. Agentic Detection Testing: Ran the newly trained version of the Agentic pipeline through a suite of structural PDFs to evaluate how well it had absorbed the previously provided .md context files. 4. Font Vulnerability Diagnosis: During testing, I discovered a significant failure mode: the Agentic AI struggled to interpret "unfamiliar" text fonts. Because the source PDFs originated from various external distributors, the lack of standardization in typography caused the agent's text-extraction layers to misread labels, severely disrupting its contextual reasoning.	full-stack pull request review workflow is completely new. Academic assignments use uniform, pre-processed datasets where typography is standardized, shielding students from the messy reality of font-driven OCR dropouts. Furthermore, school rarely requires students to step completely outside of their data workflows to audit interactive UI rendering bugs on a staging website or use advanced tools like Codex CLI to manage production-level code integration. • If not related in the school, discuss the importance of the learning that it should be taught in the school This shift highlights a vital, multi-disciplinary reality that should be integrated into tech curricula, because a data professional's work does not exist in a vacuum. A model can be theoretically perfect, but a silent upstream font parsing failure or a front-end cursor offset bug will still break the product for the end user. Teaching students how to trace a failure from user-interface coordinates down through text-extraction layers and agentic reasoning modules builds comprehensive product intuition. Introducing full-stack debugging and vulnerability testing into computer science programs creates versatile engineers who can confidently maintain the entire lifecycle of an enterprise application.
--	--	--	--

Prepared By:


Zachary Lorenzo F. Sulit
Student Trainee

Approved By:


Ardiwedhess Abad
Supervisor, Project Lead